AM Modulation

Super-Heterodyne Receiver

A Technical Report on Design and Implementation

Supervised by:

Dr. Ahmed Hesham

Presanted to:

Eng. Mohamed Khalid

Course:

EECG 232 - Introduction to Communication Systems

91240727	مروان محمد مبارك سليمان
91240630	محمد احمد محمد احمد محمود

Contents

Introduction **2**

Block-by-Block Design **2**
Signal Preprocessing . . . 2
DSB-SC Modulator & FDM Multiplexing . . . 2
RF Stage — Band-Pass Filter . . . 3
Local Oscillator & Mixer . . . 3
IF Stage — Band-Pass Filter . . . 3
Baseband Detection . . . 3

Discussion **4**
Role of RF, IF, and Baseband Stages . . . 4
Spectrum at Each Stage . . . 4
Effect of Removing the RF BPF . . . 5
Effect of Oscillator Frequency Offset . . . 6

MATLAB Source Code **7**

I Introduction

This report documents the design and simulation of a complete AM (Amplitude Modulation) transceiver system implemented in MATLAB. The modulation scheme used is DSB-SC (Double Sideband Suppressed Carrier), and the receiver architecture is the classic super-heterodyne design. Two real audio signals are used as input messages, multiplexed over a shared channel using Frequency Division Multiplexing (FDM), and subsequently demodulated by the receiver chain.

The simulation covers the following pipeline: audio preprocessing, DSB-SC modulation, FDM multiplexing, RF bandpass filtering, frequency down-conversion via a local oscillator and mixer, IF bandpass filtering, and final baseband detection with a low-pass filter. Spectrum plots are provided at each stage to verify correct operation.

II Block-by-Block Design

2.1 Signal Preprocessing

Each .wav audio file is read using audioread(), which returns a stereo (2-channel) matrix and the sampling frequency Fs. The two channels are summed to produce a mono signal:

```
[y_t, Fs] = audioread ('Short_BBCArabic2.wav');
m_t = y_t(:,1) + y_t(:,2);
```

Zero-padding is then applied to ensure both signals share the same length:

```
max_length = max(length(m_t), length(a_t));
m_t = [m_t; zeros(max_length - length(m_t), 1)];
```

Finally, upsampling by a factor of 10 is applied using interp(). This is necessary because the carrier frequencies (100 kHz, 130 kHz) exceed Fs/2 = 22 kHz of the original audio, which would violate the Nyquist criterion. Upsampling raises the effective sampling rate to 441 kHz, satisfying Nyquist for all carrier frequencies used.

```
m_t = interp(m_t, 10);
Fs  = Fs * 10;   % Fs is now 441000 Hz
```

2.2 DSB-SC Modulator & FDM Multiplexing

Each message signal is modulated using DSB-SC: the message is multiplied element-wise by a cosine carrier at the assigned frequency. DSB-SC produces no carrier component in the transmitted spectrum only the two sidebands around carrier frequency. The mathematical operation is:

```
modulated_signal_1 = m_t ' .* cos (2* pi * Fc_1 * t ) ; % DSB - SC : BBC @ 100 kHz
modulated_signal_2 = a_t ' .* cos (2* pi * Fc_2 * t ) ; % DSB - SC : FM @ 130 kHz
Tx_signal = modulated_signal_1 + modulated_signal_2 ;   % DSB - SC : FM + BBC
```

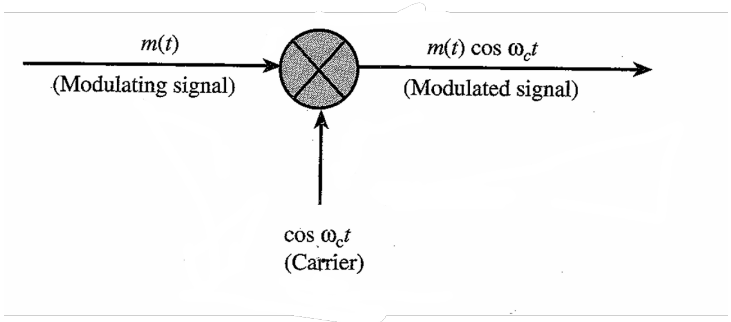

Figure 1: Transimitter (Coherent Modulator Block)

2.3 RF Stage — Band-Pass Filter

The RF stage is implemented as band-pass filter centered at the desired station carrier frequency. and each signal bandwidth is roughly estimated from frequency spectrum plot. Its role is to select only one station from the FDM composite and reject all others, including potential image-frequency interference.

```
fpass_RF            = [Fc_desired - Bw_desired Fc_desired + Bw_desired];
modulated_signal_RF = bandpass(Tx_signal, fpass_RF, Fs);
```

2.4 Local Oscillator & Mixer

The local oscillator generates a cosine at frequency $F_{lo} = F_c + F_{IF} = 115$ kHz. The mixer multiplies the RF-filtered signal by this oscillator, which frequency-shifts the desired station from 100 kHz down to the fixed intermediate frequency of 15 kHz:

```
F_lo   = Fc_desired + F_IF;
S_mixed = modulated_signal_RF .* cos(2*pi*F_osc*t);
```

After mixing, the spectrum contains two components: the desired signal at $F_{IF} = 15$ kHz and an unwanted high-frequency image at $2F_c + F_{IF} = 215$ kHz. The IF stage removes this image.

2.5 IF Stage — Band-Pass Filter

IF stage is a second BPF centered at $F_{IF} = 15$ kHz. It passes only the down-converted desired signal and removes the high-frequency image produced by the mixer as well as any residual interference:

```
fpass_IF = [F_IF - Bw_desired, F_IF + Bw_desired];
modulated_signal_IF = bandpass(S_mixed, fpass_IF, Fs);
```

2.6 Baseband Detection

The baseband detector performs the final demodulation in two steps. First, the IF signal is multiplied by a cosine at $F_{IF} = 15$ kHz. This shifts the signal spectrum from 15 kHz back to 0 Hz (baseband) but also creates a high-frequency image at $2F_{IF} = 30$ kHz. Then a low-pass filter designed with fdesign.lowpass then removes the 30 kHz image and leaves only the recovered audio signal in the baseband:

```
S_BB = S_IF .* cos(2*pi*F_IF*t);
d_lpf       = fdesign.lowpass('N,F3dB', 6, Bw_desired/(Fs/2));
hLPF        = design(d_lpf, 'butter');
Rx_signal   = filter(hLPF, S_BB);
```

The specification string **'N,F3dB'** tells **fdesign.lowpass** that the filter is defined by its order (N=6) and its 3 dB cutoff frequency, which is normalized to the range [0,1] by dividing by $F_s/2$.

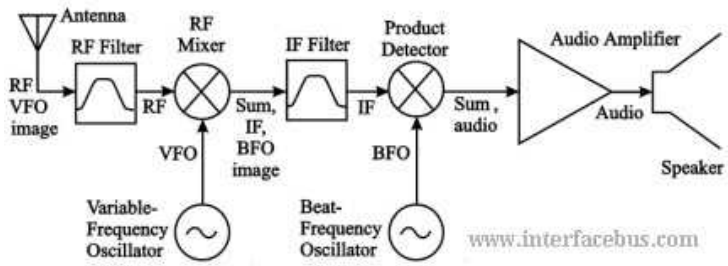

Figure 2: Super heterodyne receiver block

III Discussion

i. Role of RF, IF, and Baseband Stages

The RF stage is a tunable band-pass filter at the antenna. Its primary role is image-frequency rejection: it selects only the desired station carrier and suppresses all other stations and interference before the signal reaches the mixer.
The IF stage is a fixed band-pass filter at the intermediate frequency (15 kHz in this simulation). Because the IF frequency is always constant regardless of which station is selected, the IF filter can be designed once with very tight, optimized characteristics. This is the key advantage of the super-heterodyne architecture: selectivity and sensitivity are achieved at a fixed, low IF rather than at the variable RF carrier.
The baseband detector performs the final down-conversion from F_{IF} to 0 Hz and applies a low-pass filter to recover the original audio message. Without this stage the signal would remain modulated at the intermediate frequency and would not be audible.

Why do we need the IF stage? Designing a sharp, stable BPF at RF frequencies (100 kHz, 130 kHz) is difficult because small component tolerances cause large frequency errors at high frequencies. By first converting the signal to a lower, fixed IF, the demanding filtering work is done at 15 kHz where filters are far easier to build and keep stable.

ii. Spectrum at Each Stage (Demodulating Station 1)

The following describes the expected spectral content observed at the output of each stage when demodulating Station 1 (BBC Arabic at $F_c = 100$ kHz):

- **After RF BPF** The FDM composite spectrum contains two pairs of sidebands at ±100 kHz and ±130 kHz. After the RF BPF (passband 92–108 kHz), only the sideband pair at ±100 kHz remains. The 130 kHz station is fully suppressed.
- **After IF BPF** After mixing with the 115 kHz oscillator, the 100 kHz signal shifts to 15 kHz. A high-frequency image also appears at approximately 215 kHz. The IF BPF (passband 7–23 kHz) retains only the 15 kHz component and removes the image.
- **After Baseband Detection** Multiplying by $cos(2\pi 15000t)$ shifts the IF signal to baseband (0 Hz). The LPF removes the 30 kHz image, leaving a clean spectrum centered at 0 Hz with bandwidth equal to the original audio signal (approximately 8 kHz for speech).

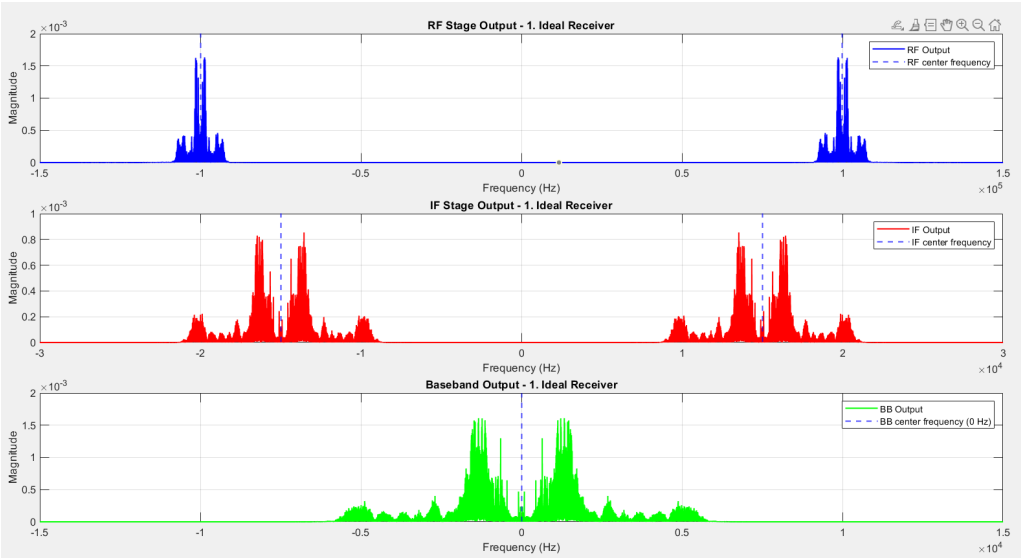

Figure 3: Ideal Receiver Spectrum

iii. Effect of Removing the RF BPF

When the RF BPF is removed both stations land at the same IF frequency. The IF BPF cannot distinguish between them because they now fully overlap in the spectrum. The result at baseband is a superposition of both audio streams, making the recovered audio heard simultaneously and interfere with each other.

This clearly demonstrates the critical role of the RF stage: it performs image-frequency rejection, ensuring that only the desired station reaches the mixer.

RF Stage (No BPF Applied)

Without the RF band-pass filter, the full FDM composite signal containing both stations at 100 kHz and 130 kHz passes directly into the mixer. As seen in Figure 4, both sidebands are clearly present with equal strength — neither station has been rejected.

IF Stage Output

After mixing with the local oscillator at $F_c + F_{IF}$, both stations are down-converted and land at the same intermediate frequency of 15 kHz. Figure 5 shows that the two signals fully overlap at the IF, making it impossible for the IF BPF to separate them. The filter passes both signals together, as they are indistinguishable at this stage.

Recovered Baseband Signal

At the baseband detection output, Figure 6 shows a corrupted spectrum where the energy of both audio stations is superimposed across the entire baseband band. Unlike the clean single-lobe spectrum obtained with the RF BPF active, this spectrum contains the mixed content of both stations. When played back using `sound()`, the output is unintelligible — both broadcasts are audible simultaneously with heavy mutual interference.

Conclusion of this experiment: The RF band-pass filter is not optional. Without it, the superheterodyne receiver loses its ability to tune to a single station. Any signal whose frequency satisfies $|F - F_{lo}| = F_{IF}$ will alias directly onto the desired channel at the IF stage, an effect known as *image-frequency interference*. The RF BPF is the only stage that can reject this image before it corrupts the received signal.

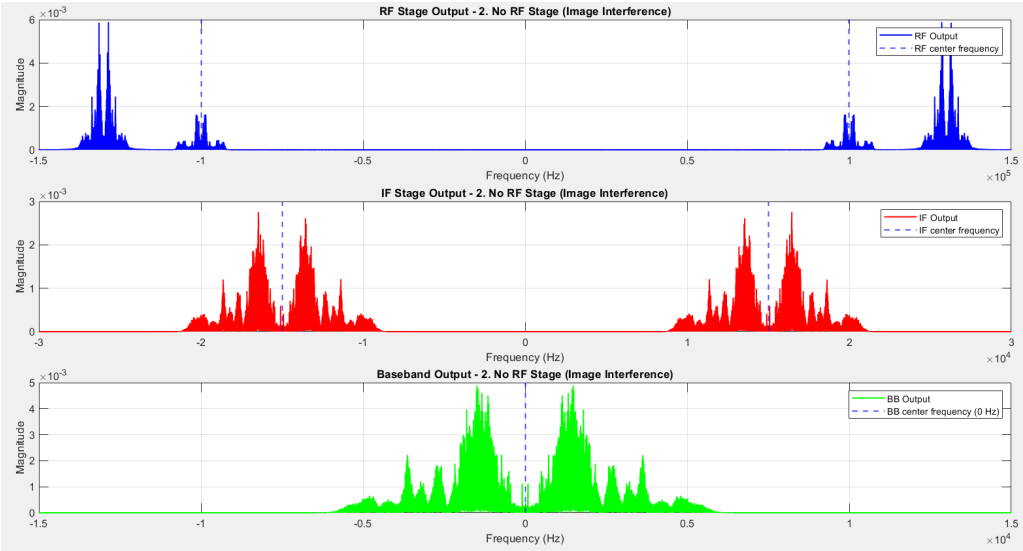

Figure 4: spectrum without RF BPF

iv. Effect of Oscillator Frequency Offset

Small Offset (0.1 kHz)

In this case, the signal shifts from 15 kHz to 15.1 kHz.

Spectrum All frequency components move to the right by 100 Hz.

Audio The speech is still clear, but the pitch is slightly higher. The voice sounds a little different (change in timbre), but you can still understand everything easily.

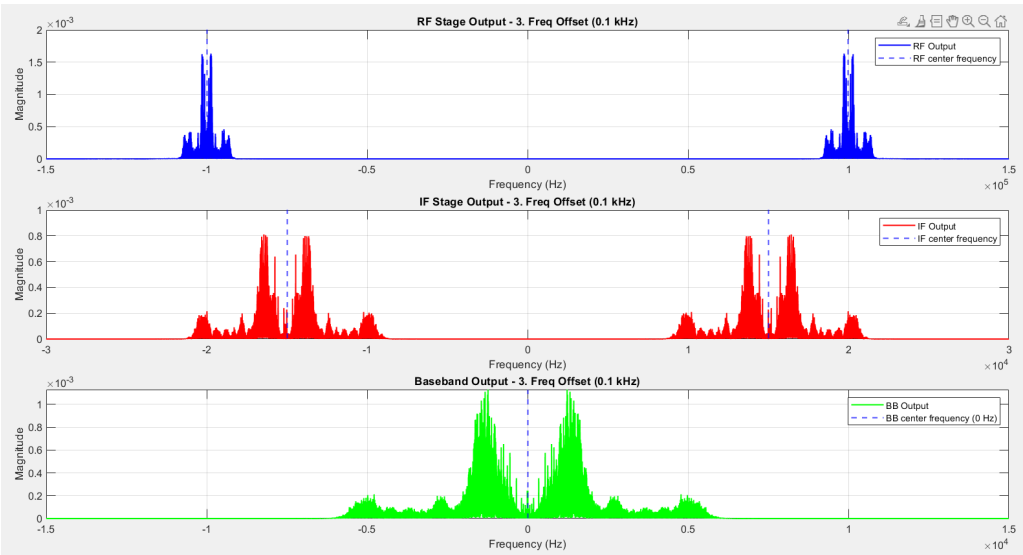

Figure 5: spectrum with 100 Hz Phase Offset

Large Offset (1 kHz)

Here, the signal shifts much further to 16 kHz.

Spectrum There is a large 1 kHz shift to the right. Because of this, the Low-Pass Filter (LPF) may cut off the higher parts of the signal.

Audio The voice sounds very unnatural and robotic. It is much harder to understand the speech because the frequencies are pushed too far from their original place.

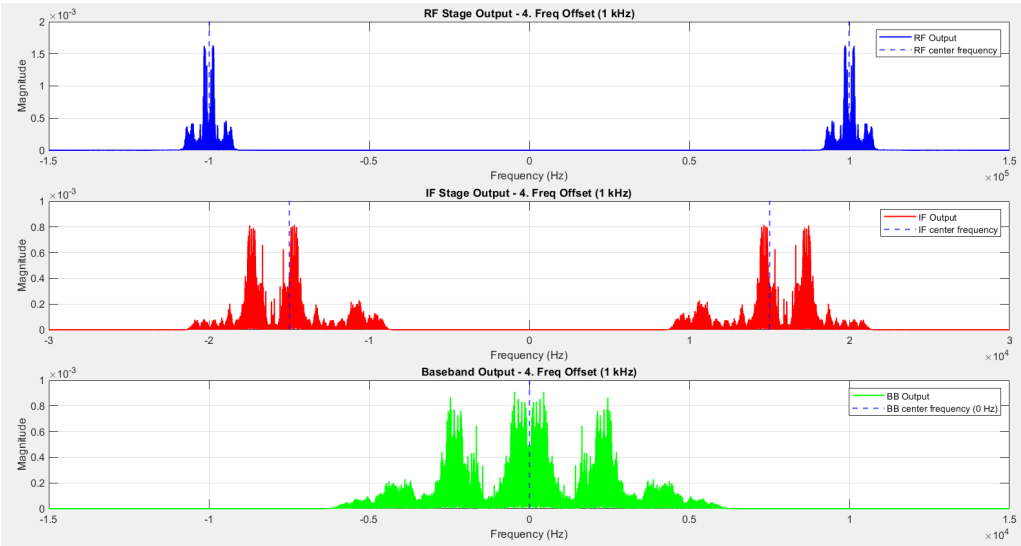

Figure 6: spectrum with 1kHZ Phase Offset

Conclusion

This experiment proves that the Local Oscillator must be very stable. If the frequency is even slightly wrong, the output audio changes. In real-world receivers, we use a Phase-Locked Loop (PLL) like Costas Loop to "lock" the frequency and make sure the local oscillator stays exactly on the correct frequency.

IV MATLAB Source Code

```
% Amplitude Modulation - Super Heterodyne Receiver
% Author : Marwan Mobarak
% Date   : 4/3/2026
%% ===================== File Processing ================================
[y_t, Fs1] = audioread("Short_BBCArabic2.wav");
m_t = y_t(:,1) + y_t(:,2);                      % Stereo to Mono
[z_t, Fs2] = audioread("Short_FM9090.wav");
a_t = z_t(:,1) + z_t(:,2);                      % Stereo to Mono
Fs = max(Fs1, Fs2);                             % Use higher sampling rate
clear y_t z_t Fs1 Fs2;
%% ===================== Zero Padding ===================================
max_length = max(length(m_t), length(a_t));
m_t = [m_t; zeros(max_length - length(m_t), 1)];
a_t = [a_t; zeros(max_length - length(a_t), 1)];
%% ===================== Upsampling =====================================
upsample_factor = 10;
m_t        = interp(m_t, upsample_factor);      % Interpolate signal 1
a_t        = interp(a_t, upsample_factor);      % Interpolate signal 2
Fs         = Fs * upsample_factor;              %update sampling frequency
Ts         = 1/Fs;
max_length = length(m_t);
%% ============== Plot Signals in Frequency Domain ======================
figure('Name','Frequency Domain - Messages','NumberTitle','off');
```

```
subplot(2,1,1);
N1 = length(m_t);
f1 = (0:N1-1) * (Fs/N1);
plot(f1(1:floor(N1/2)), abs(fft(m_t(1:floor(N1/2))))/N1, 'b', 'LineWidth', 1.2);
xlabel('Frequency (Hz)'); ylabel('Magnitude');
title(sprintf('BBC Arabic - Baseband Spectrum (Fs = %d Hz)', Fs));
xlim([0 10000]); grid on;
subplot(2,1,2);
N2 = length(a_t);
f2 = (0:N2-1) * (Fs/N2);
plot(f2(1:floor(N2/2)), abs(fft(a_t(1:floor(N2/2))))/N2, 'r', 'LineWidth', 1.2);
xlabel('Frequency (Hz)'); ylabel('Magnitude');
title(sprintf('FM 90.90 - Baseband Spectrum (Fs = %d Hz)', Fs));
xlim([0 10000]); grid on;
%% ======================== AM Modulator (DSB-SC) ========================
Fc_1 = 100000;                      % Carrier frequency for signal 1 (100 kHz)
Fc_2 = 130000;                      % Carrier frequency for signal 2 (130 kHz)
Bw_1 = 5000;                        % Bandwidth of signal 1 (5 kHz)
Bw_2 = 5000;                        % Bandwidth of signal 2 (5 kHz)
t    = (0 : max_length-1) * Ts;     % Create time vector for modulation


modulated_signal_1 = m_t' .* cos(2*pi*Fc_1*t);   % Multiply message 1 by carrier 1
modulated_signal_2 = a_t' .* cos(2*pi*Fc_2*t);   % Multiply message 2 by carrier 2

% Combine both signals into a single transmission channel (Frequency Division Multiplexing)
Tx_signal  = modulated_signal_1 + modulated_signal_2;

%% ====================== initializing the loop ========================
% Test 4 different receiver scenarios: ideal, missing RF filter, small freq error, large freq error
Freq_offset_LIST = [0, 0, 100, 1000];
Titles = {'1. Ideal Receiver', '2. No RF Stage (Image Interference)', ...
          '3. Freq Offset (0.1 kHz)', '4. Freq Offset (1 kHz)'};
for i = 1 : 4
    Freq_offset = Freq_offset_LIST(i);

    %% ======================== RF Stage ================================
    Fc_desired  = Fc_1;
    Bw_desired  = Bw_1;
    fpass_RF    = [Fc_desired - Bw_desired, Fc_desired + Bw_desired];

    if i ~= 2  % If it's NOT the 2nd iteration, use the filter
        modulated_signal_RF = bandpass(Tx_signal, fpass_RF, Fs);
    else       % If it IS the 2nd iteration, BYPASS the filter
        modulated_signal_RF = Tx_signal;
    end
    %% ======================== Mixer ===================================
    F_IF        = 15000;                            % Intermediate Frequency (15 kHz)

    % Calculate Local Oscillator frequency (High-side injection: f_LO = f_C + f_IF + error)
    F_lo       = Fc_desired + F_IF + Freq_offset;

    S_mixed     = modulated_signal_RF .* cos(2*pi*F_lo*t);


    %% ======================== IF Stage ================================
    fpass_IF    = [F_IF - Bw_desired, F_IF + Bw_desired];
    modulated_signal_IF        = bandpass(S_mixed, fpass_IF, Fs);


    %% ======================== Baseband Detection ======================
    S_BB        = modulated_signal_IF .* (2 * cos(2*pi*F_IF*t));

    d_lpf       = fdesign.lowpass('N,F3dB', 6, Bw_desired/(Fs/2));
    hLPF        = design(d_lpf, 'butter');
    Rx_signal = filter(hLPF, S_BB);


%% ======================== Listen & Plot ============================
    figure('Name', Titles{i}, 'NumberTitle', 'off'); % Dynamic Window Title
```

```
    % Setup frequency axis for all plots
    N   = length(Rx_signal);
    f   = (-N/2 : N/2-1) * (Fs/N);

    % 1. Plot RF Stage Output (Centered at 100 kHz)
    subplot(3,1,1);
    plot(f, abs(fftshift(fft(modulated_signal_RF)))/N, 'b', 'LineWidth', 1.2, 'DisplayName', 'RF Output');
    hold on;
    xline(Fc_desired, '--b', 'LineWidth', 1.2, 'DisplayName', 'RF center frequency');
    xline(-Fc_desired, '--b', 'LineWidth', 1.2, 'HandleVisibility', 'off'); % Hide from legend
    hold off;
    xlabel('Frequency (Hz)'); ylabel('Magnitude');
    title(['RF Stage Output - ', Titles{i}]);
    xlim([-150000 150000]); % Zoomed out to see the 100kHz and 130kHz carriers
    grid on; legend('show');

    % 2. Plot IF Stage Output (Centered at 15 kHz)
    subplot(3,1,2);
    plot(f, abs(fftshift(fft(modulated_signal_IF)))/N, 'r', 'LineWidth', 1.2, 'DisplayName', 'IF Output');
    hold on;
    xline(F_IF, '--b', 'LineWidth', 1.2, 'DisplayName', 'IF center frequency');
    xline(-F_IF, '--b', 'LineWidth', 1.2, 'HandleVisibility', 'off'); % Hide from legend
    hold off;
    xlabel('Frequency (Hz)'); ylabel('Magnitude');
    title(['IF Stage Output - ', Titles{i}]);
    xlim([-30000 30000]); % Zoomed in to see the 15kHz IF signal
    grid on; legend('show');

    % 3. Plot Baseband Output (Centered at 0 Hz)
    subplot(3,1,3);
    plot(f, abs(fftshift(fft(Rx_signal)))/N, 'g', 'LineWidth', 1.2, 'DisplayName', 'BB Output');
    hold on;
    xline(0, '--b', 'LineWidth', 1.2, 'DisplayName', 'BB center frequency (0 Hz)');
    hold off;
    xlabel('Frequency (Hz)'); ylabel('Magnitude');
    title(['Baseband Output - ', Titles{i}]);
    xlim([-15000 15000]); % Zoomed in to see the baseband audio
    grid on; legend('show');

    % Audio Playback
    Fs_play     = 44100;
    m_play      = Rx_signal(1:upsample_factor:end);

    disp(['Playing: ', Titles{i}]);
    sound(m_play, Fs_play);
    pause(length(m_play)/Fs_play + 1);
end
```